# NIET BUSINESS SCHOOL

Post Graduate Diploma in Management (PGDM)

**BUSINESS ANALYTICS & DATA ANALYTICS | PROJECT REPORT**

**PROJECT TITLE**

# Customer Churn Analysis and Prediction for a Telecom Company



| | |
|---|---|
| **Submitted by** | **Abhishek Kumar** |
| **Roll Number** | **252737000** |
| **Section** | **B** |
| **Programme** | **PGDM** |
| **College** | **NIET Business School** |
| **Submission** | **October 2026** |



**Tools: Python | Pandas | NumPy | Matplotlib | Scikit-learn | Git & GitHub**

# Table of Contents


| No. | Section |
|---|---|
| 1 | Title and Introduction |
| 2 | Problem Statement, Objectives, Purpose and Scope |
| 3 | Tools Used, Project Structure, Architecture Diagram and Working Mechanism |
| 4 | Dataset Description |
| 5 | Work Done (Step-by-Step Methodology) |
| 6 | Exploratory Data Analysis and Data Interpretation |
| 7 | Churn Prediction Model and Interpretation |
| 8 | Retention Campaign Scenario and Recommendations |
| 9 | Uses of the Project |
| 10 | Applications |
| 11 | Limitations |
| 12 | Conclusion and Future Scope |
| 13 | GitHub Deployment |
| 14 | References |
| A | Appendix - Complete Source Code |

# 1. Title and Introduction

## 1.1 Project Title

Customer Churn Analysis and Prediction for a Telecom Company

## 1.2 Introduction

Churn means a customer stops using a company's service. For subscription businesses such as telecom, broadband and OTT, winning a new customer usually costs much more than keeping an existing one, so every customer who leaves is a direct loss of future revenue. Business analytics helps a company answer three questions: who is leaving, why are they leaving, and which customers should be contacted first with a retention offer.

This project analyses 5,000 telecom customers. It cleans the data, studies how churn changes with contract type, tenure, internet service, support calls, payment method and charges, builds three machine-learning models to predict which customers will churn, identifies the main drivers of churn, and finally converts the model into a business decision by estimating the profit of a targeted retention campaign.

**Important note on data:** the dataset is simulated. It was generated by a seeded Python script (generate_data.py) in which churn depends on realistic drivers, so that the full analytics pipeline can be demonstrated and reproduced. The method is reusable: replacing the CSV with a real customer export from a CRM or billing system will run the same pipeline. Findings describe this dataset and should not be read as real market statistics.

# 2. Problem Statement, Objectives, Purpose and Scope

## 2.1 Problem Statement

A telecom company loses 26.2% of its customers, which is about Rs. 143.3 lakh of yearly billing in this dataset, but it does not know which customers are about to leave or why. Contacting every customer with an offer is expensive. The question this project answers is: which customer groups churn most, what drives churn, can churn be predicted in advance, and which customers should be targeted to make a retention campaign profitable?

## 2.2 Objectives

- To clean the customer data and calculate the churn rate.
- To find how churn varies with contract, tenure, services, support calls, payment method and charges.
- To build and compare churn prediction models using standard evaluation measures.
- To identify the most important drivers of churn.
- To estimate the profit of a targeted retention campaign and decide how many customers to contact.
- To package the project with documentation for deployment on GitHub.

## 2.3 Purpose

The purpose is to turn customer records into decisions: who to protect, what to fix and where to spend retention budget. It also demonstrates the complete analytics lifecycle: data preparation, exploratory analysis, predictive modelling, evaluation, business interpretation and deployment.

## 2.4 Scope

The scope covers 5,000 customers, 14 predictor columns and the churn status. Customer lifetime value is approximated by twelve months of billing. Network quality data, competitor offers and customer satisfaction surveys are outside the scope.

# 3. Tools, Structure, Architecture and Mechanism

## 3.1 Tools and Technologies Used

| Tool / Library | Purpose in this project |
|---|---|
| Python 3 | Main programming language for the entire pipeline |
| Pandas | Loading the CSV, cleaning, grouping and summarising |
| NumPy | Simulated data generation and numerical operations |
| Matplotlib | All charts and diagrams |
| Scikit-learn | Train/test split, scaling, Logistic Regression, Random Forest, Gradient Boosting and evaluation metrics |
| Git and GitHub | Version control and public hosting of the project |
| ReportLab | Generation of this PDF report |

## 3.2 Project Structure (Repository Layout)

```
customer-churn-analysis/
|-- data/
|   |-- telecom_churn.csv          raw dataset (5,000 customers)
|   `-- telecom_churn_clean.csv    cleaned data with extra columns
|-- src/
|   |-- generate_data.py           builds the dataset
|   |-- analysis.py                cleaning, EDA, models, evaluation, campaign profit, charts
|   |-- diagrams.py                architecture / mechanism diagrams
|   `-- build_report.py            builds the PDF report
|-- outputs/
|   |-- charts/                    all figures (PNG)
|   `-- results.json               metrics and summary numbers
|-- docs/
|   `-- Customer_Churn_Analysis_Report.pdf
|-- requirements.txt
|-- .gitignore
`-- README.md
```

## 3.3 System Architecture Diagram

Fig 3.1 System Architecture / Workflow of the Project

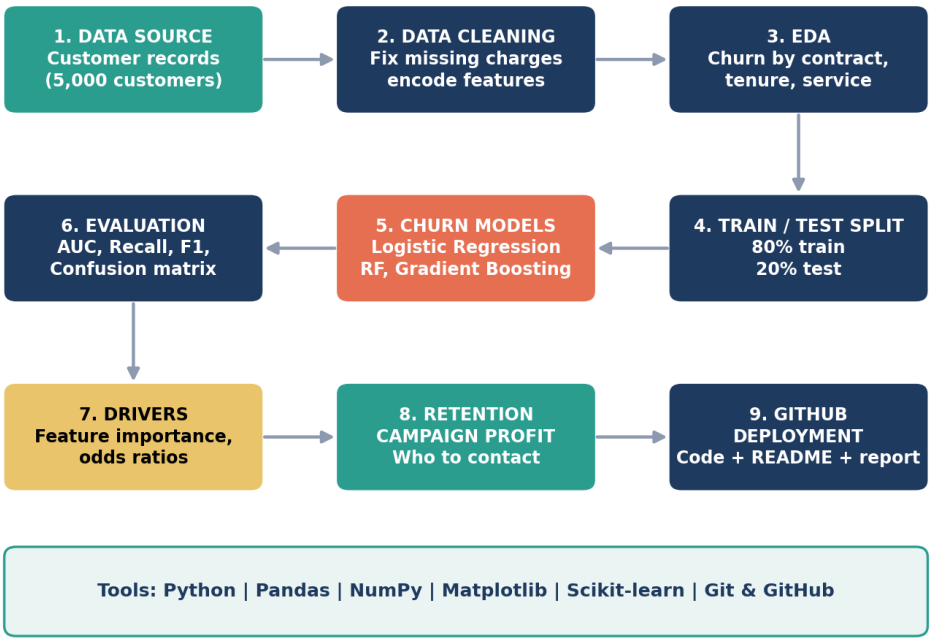



*Fig 3.1 System architecture / workflow of the project*

The pipeline runs from raw customer data to cleaning, exploratory analysis, model training, evaluation, driver analysis, campaign profit estimation and deployment.

## 3.4 Working Mechanism

Fig 3.2 Working Mechanism: From Risk Score to Retention Profit

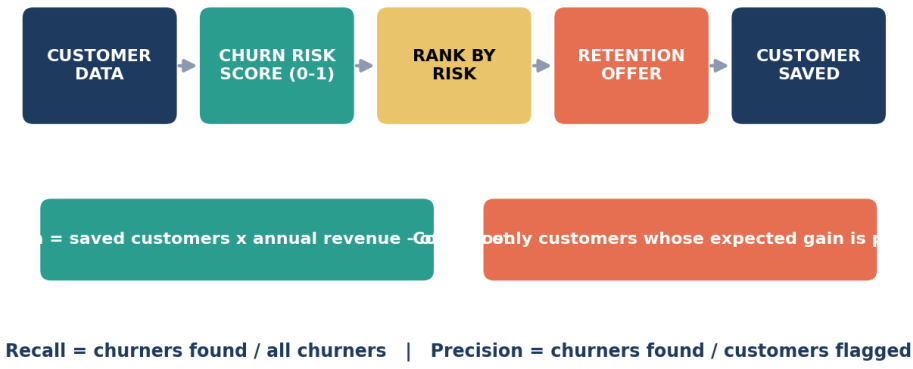



*Fig 3.2 Working mechanism: from risk score to retention profit*

**How the mechanism works.** The trained model gives every customer a churn risk score between 0 and 1. Customers are ranked from highest to lowest risk. A retention offer (a discount, a plan upgrade or a call from the retention team) costs money, so it should be sent only where the expected benefit is larger than the cost. The expected benefit is the chance of saving the customer multiplied by the revenue the customer would otherwise take away.

| Measure | Formula | Meaning |
|---|---|---|
| Churn rate | Churned customers / All customers x 100 | Share of customers lost |
| Accuracy | Correct predictions / All predictions | Overall correctness |
| Precision | True churners flagged / All customers flagged | How reliable a flag is |
| Recall | True churners flagged / All real churners | How many churners are found |
| F1-score | 2 x Precision x Recall / (Precision + Recall) | Balance of precision and recall |
| ROC-AUC | Area under the ROC curve | Ability to rank churners above stayers (0.5 = random, 1 = perfect) |

# 4. Dataset Description

The dataset has 5,000 customers and 17 columns (one row per customer). Extra columns (churn flag and tenure group) are added during analysis.

| Column | Type | Description |
|---|---|---|
| customer_id | Text | Unique customer ID (CUS10000 onwards) |
| gender, senior_citizen, partner, dependents | Category | Customer profile |
| tenure_months | Number | Months the customer has been with the company |
| contract | Category | Month-to-month / One year / Two year |
| internet_service | Category | DSL / Fiber optic / No |
| online_security, tech_support, streaming | Category | Add-on services (Yes / No / No internet) |
| payment_method | Category | Electronic check / Mailed check / Bank transfer / Credit card |
| paperless_billing | Category | Yes / No |
| support_calls | Number | Customer-care calls in the recent period |
| monthly_charges | Number | Monthly bill in INR |
| total_charges | Number | Total billed so far in INR |
| churn | Target | Yes if the customer left, No if the customer stayed |

## 4.1 Data Quality and Cleaning

The raw file had 9 missing values in monthly_charges and 14 missing values in total_charges. Missing monthly charges were filled with the median of the customer's internet-service group, because the bill depends strongly on the service type. Missing total charges were rebuilt as monthly charges multiplied by tenure. No duplicate customer IDs were found.

## 4.2 Overall Summary

| Metric | Value | Metric | Value |
|---|---|---|---|
| Customers | 5,000 | Churned customers | 1,308 |
| Churn rate | 26.2% | Average tenure | 29.9 months |
| Average monthly bill | Rs. 767 | Monthly billing lost to churn | Rs. 11.9 lakh |
| Yearly billing at risk | Rs. 143.3 lakh | Train / test split | 4,000 / 1,000 |

# 5. Work Done (Step-by-Step Methodology)

## Step 1 - Topic selection and problem definition

Customer churn was chosen because it is one of the most common business-analytics problems and has a clear financial meaning.

## Step 2 - Data creation

A seeded script simulates 5,000 customers. Contract, services, payment method and support calls are drawn with realistic proportions, and churn is generated from a logistic equation in which month-to-month contracts, short tenure, fiber service, electronic-check payment, more support calls and no tech support raise the churn probability. The intercept is tuned so that overall churn is close to 26%.

## Step 3 - Data cleaning

Missing charges were filled from service-group medians and tenure (Section 4.1). Tenure was also grouped into 0-12, 13-24, 25-48 and 49-72 months.

## Step 4 - Exploratory analysis

Churn rate was calculated for every important feature and six charts were produced (Section 6).

## Step 5 - Model building

Categorical columns were one-hot encoded. The data was split 80/20 with stratification so that the churn share is the same in both parts.

```
X = pd.get_dummies(df[feat], drop_first=True).astype(float)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
models = {'Logistic Regression': make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
          'Random Forest': RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=10, random_state=4
          'Gradient Boosting': GradientBoostingClassifier(random_state=42)}
```

## Step 6 - Evaluation

Models were compared on the unseen test customers using accuracy, precision, recall, F1-score and ROC-AUC. The model with the highest ROC-AUC was selected (Section 7).

## Step 7 - Business scenario

The selected model's risk scores were used to rank customers and to calculate the profit of a retention campaign for different numbers of contacted customers (Section 8).

## Step 8 - Documentation and deployment

README, requirements and .gitignore were prepared so the project can be pushed to GitHub (Section 13).

# 6. Exploratory Data Analysis and Data Interpretation

## 6.1 Overall Churn and Contract Type

Fig 4.1 Overall Churn and Contract Type

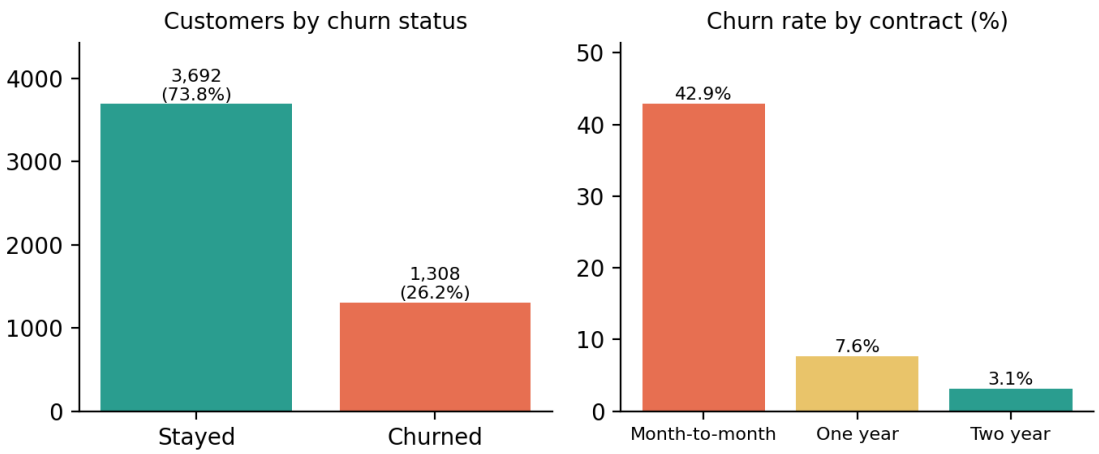



*Fig 4.1 Overall churn and churn rate by contract type*

Out of 5,000 customers, 1,308 (26.2%) left. Contract type is the clearest dividing line: month-to-month customers churn at 42.9%, compared with 7.6% for one-year and only 3.1% for two-year contracts. A month-to-month customer is about 14 times as likely to leave as a two-year customer, because there is no commitment holding them.

## 6.2 Tenure

Fig 4.2 Churn Rate by Customer Tenure

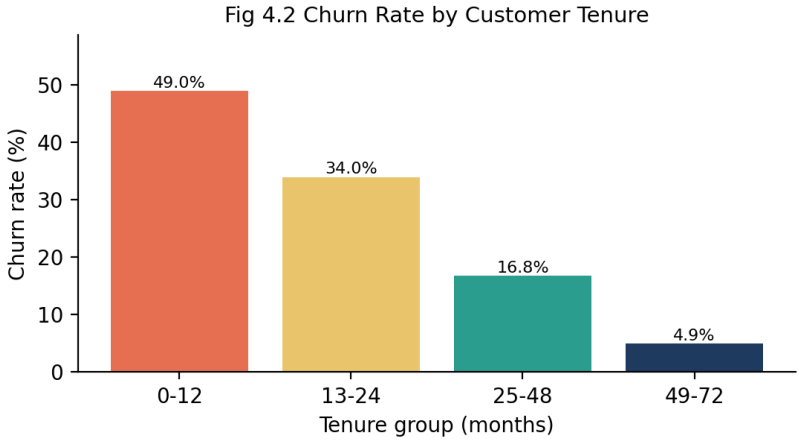



*Fig 4.2 Churn rate by customer tenure*

Churn falls steadily with tenure: 49.0% in the first year, 34.0% in the second year and 4.9% for customers older than four years. The first twelve months are the most dangerous period, so onboarding and early-life offers have the highest return.

## 6.3 Internet Service and Tech Support

Fig 4.3 Churn Rate by Service Features

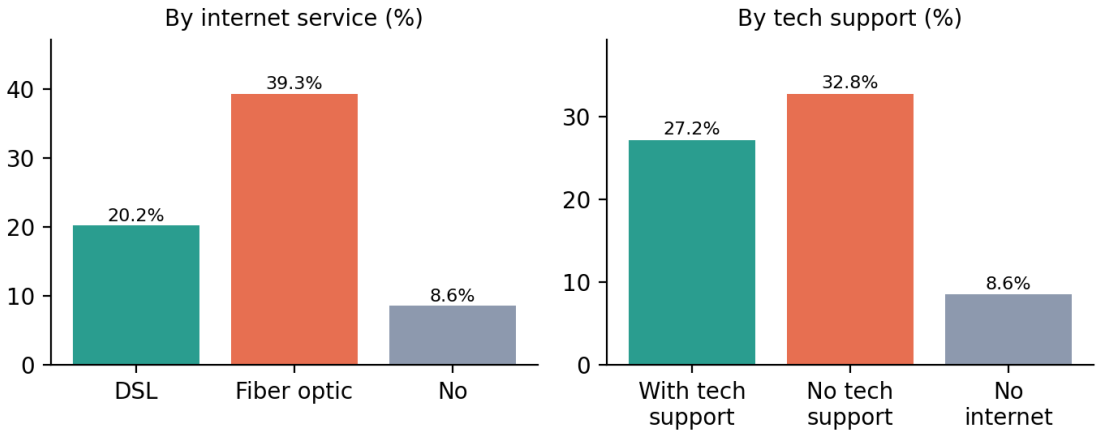



*Fig 4.3 Churn rate by internet service and tech support*

Fiber-optic customers churn at 39.3%, almost twice the 20.2% of DSL customers, even though they pay the highest bills; price or service problems on the premium plan are a likely cause. Customers with internet but without tech support churn at 32.8% versus 27.2% with tech support. Customers without internet churn least (8.6%), since they have a simple, cheap phone-only plan.

## 6.4 Monthly Charges

Fig 4.4 Monthly Charges: Churned vs Stayed

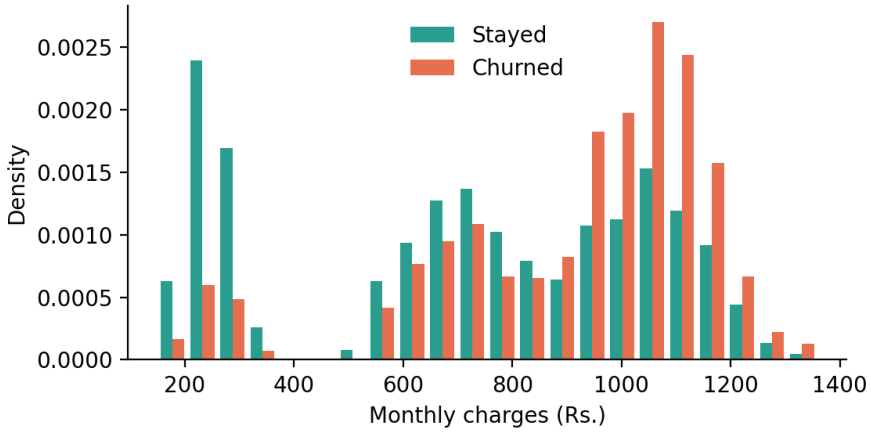



*Fig 4.4 Distribution of monthly charges for churned and stayed customers*

Churned customers pay more on average (Rs. 913 per month) than customers who stayed (Rs. 715). High bills are partly a result of the fiber plans, so price alone should not be blamed; price and service quality act together.

## 6.5 Support Calls and Payment Method

Fig 4.5 Churn Rate by Support Calls and Payment Method

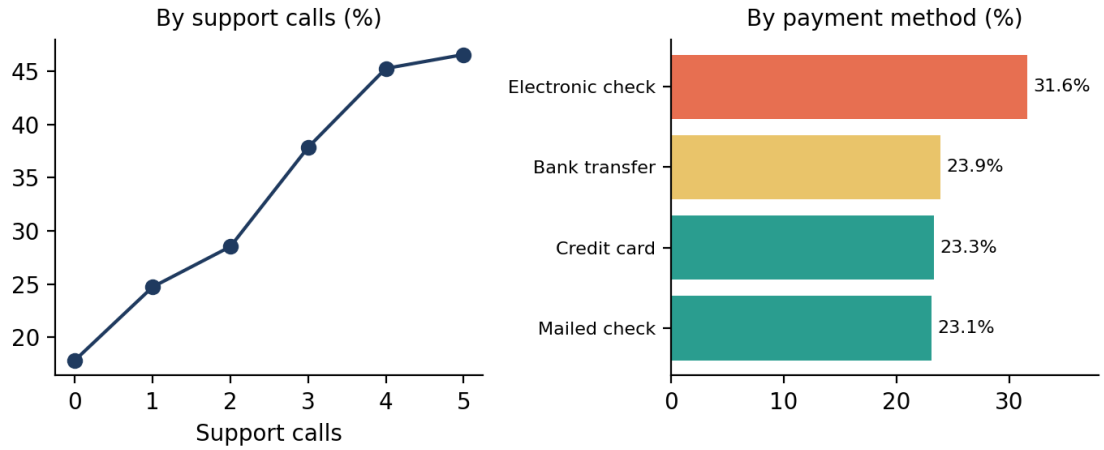



*Fig 4.5 Churn rate by number of support calls and by payment method*

Each extra support call is a warning sign: churn is 17.8% for customers with no calls, 37.8% for three calls and 45.3% for four calls. Customers paying by electronic check churn at 31.6%, against about 23% for bank transfer, credit card and mailed cheque; automatic payment customers are more settled.

## 6.6 Correlation Matrix

Fig 4.6 Correlation Matrix (numeric columns)

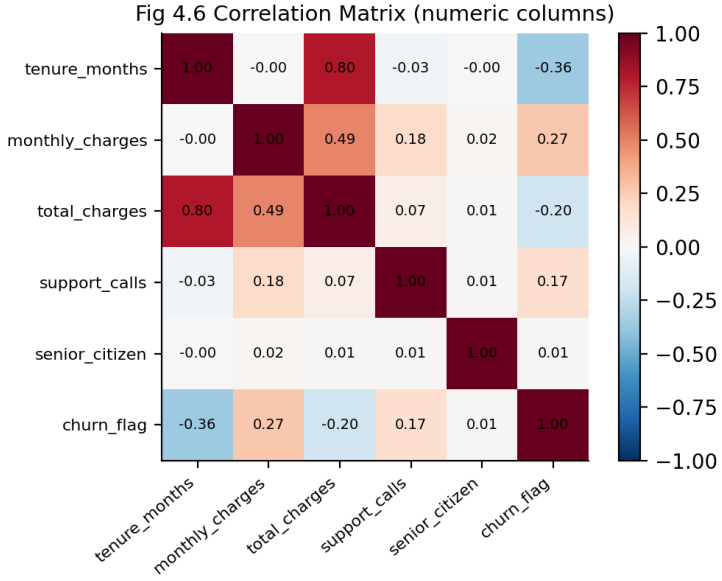



*Fig 4.6 Correlation matrix of numeric columns*

Tenure has the strongest negative correlation with churn (-0.36): longer-staying customers are far less likely to leave. Support calls (0.17) and monthly charges (0.27) are positively related to churn. Total charges is strongly linked to tenure because it is monthly charges accumulated over time, so it is not used as a predictor.

# 7. Churn Prediction Model and Interpretation

Three models were trained on 4,000 customers and tested on 1,000 unseen customers. A model that simply predicts that nobody churns would be 73.8% accurate, so accuracy alone is not enough; recall and ROC-AUC matter more.

| Model | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.81 | 0.675 | 0.531 | 0.594 | 0.854 |
| Random Forest | 0.793 | 0.667 | 0.42 | 0.515 | 0.847 |
| Gradient Boosting | 0.803 | 0.663 | 0.504 | 0.573 | 0.847 |

**Selected model: Logistic Regression** (ROC-AUC = 0.854). It ranks a random churner above a random stayer about 85% of the time, and at the default 0.5 cut-off it finds 53% of churners, with 68% of its churn flags being correct.

**Why Logistic Regression wins here.** In this simulated data the drivers of churn add up in a smooth way, which is exactly what logistic regression assumes, so a simple model matches or beats the complex ones and is also easy to explain. On real customer data, where effects interact, tree-based models often do better; the pipeline compares them automatically, so the best model is always chosen from the numbers.

## 7.1 ROC Curves and Confusion Matrix

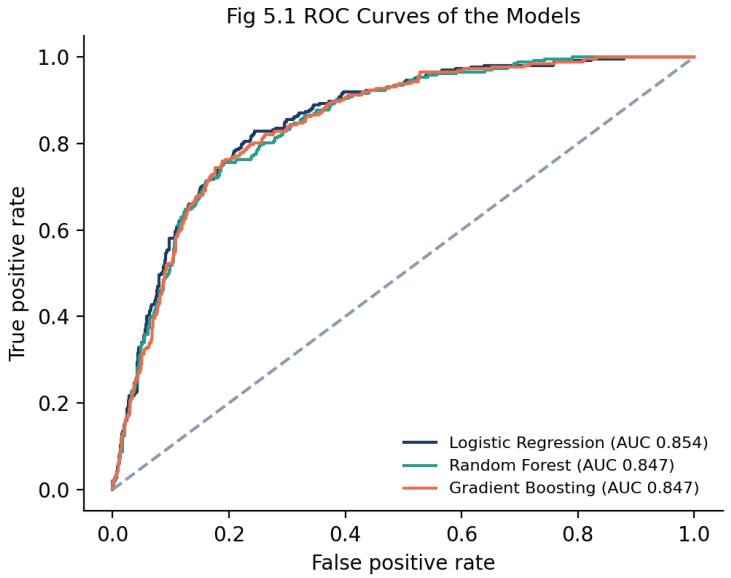



*Fig 5.1 ROC curves of the three models*

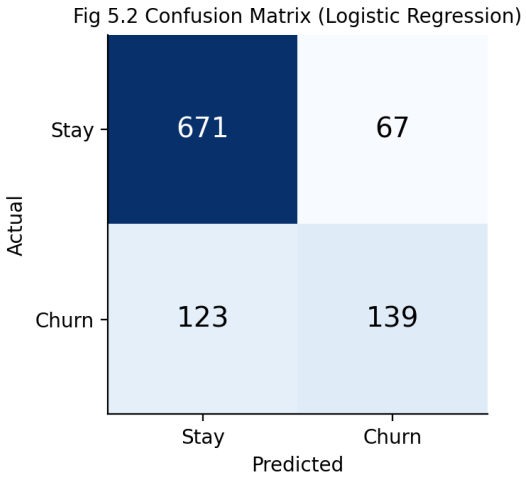



*Fig 5.2 Confusion matrix of Logistic Regression on the test customers*

Of 1,000 test customers, the model correctly identified 139 churners and 671 stayers. It missed 123 churners (false negatives) and wrongly flagged 67 stayers (false positives). For retention, missing a churner is usually more costly than contacting a customer who would have stayed, so the cut-off can be lowered to catch more churners; Section 8 chooses it using profit.

## 7.2 What Drives Churn

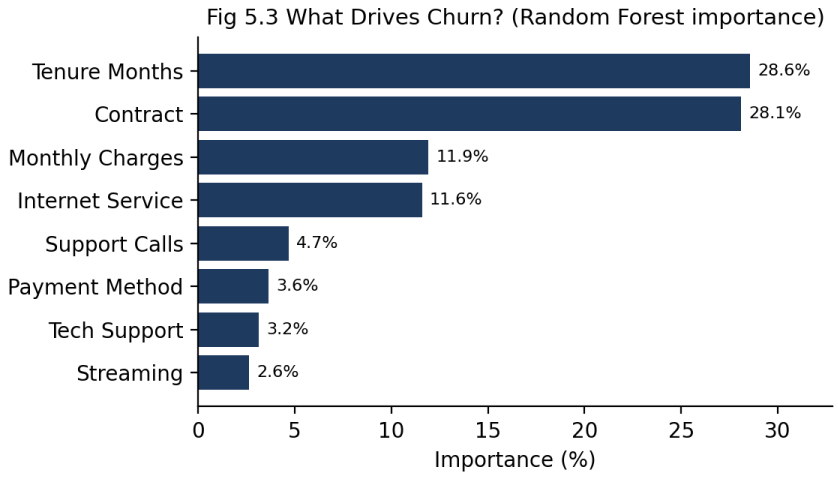



*Fig 5.3 Importance of each feature group (Random Forest)*

Tenure (28.6%) and contract type (28.1%) together explain more than half of the model's decisions, followed by monthly charges (11.9%) and internet service (11.6%). Support calls, payment method and tech support have smaller but real effects.

Logistic-regression odds ratios agree: a two-year contract multiplies the odds of churn by 0.31 and a one-year contract by 0.44 (compared with month-to-month), fiber-optic service raises the odds by 1.86 times, electronic-check payment by 1.34 times, and one more standard deviation of support calls by 1.41 times. One standard deviation of tenure multiplies the odds by 0.5.

# 8. Retention Campaign Scenario and Recommendations

**Scenario:** a retention offer costs Rs. 500 per customer contacted. It is assumed to save a churning customer with a probability of 30%, and a saved customer is worth twelve months of billing. Test customers are ranked by predicted churn risk and contacted from the highest risk downwards. Net profit = (saved customers x yearly billing) - (customers contacted x Rs. 500).

## 8.1 Scenario Result

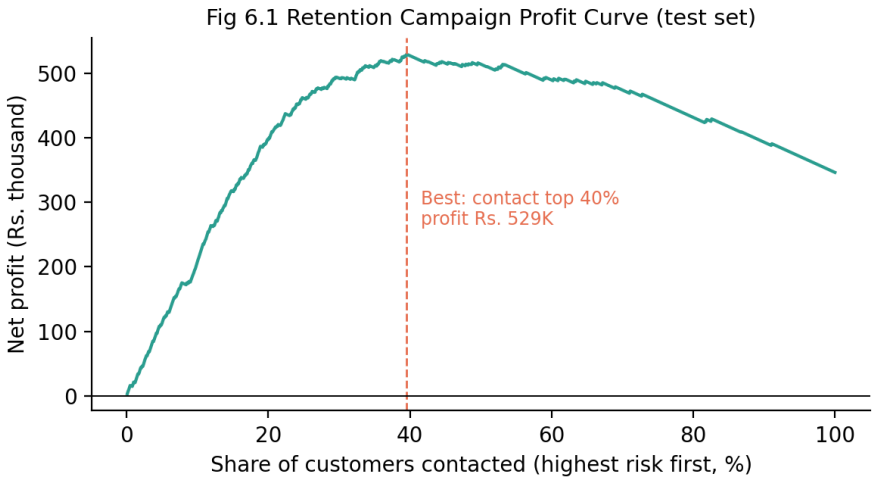



*Fig 6.1 Net profit of the retention campaign by share of customers contacted*

Profit rises while the most risky customers are contacted, peaks, and then falls as low-risk customers (who would not have left anyway) are added. The best choice is to contact the top 40% of customers (395 of 1,000), which reaches 216 of the 262 churners and gives a net profit of about Rs. 529 thousand on the test group. Contacting everyone would give only Rs. 346 thousand, and doing nothing gives zero. Targeting by risk score therefore raises campaign profit by about 53% over a blanket campaign. This is an estimate based on the assumed cost and success rate, not a guarantee.

## 8.2 Recommendations

- Move month-to-month customers to one- or two-year plans with a discount or a free add-on; this is the single largest lever.
- Build a first-year programme (welcome call, usage tips, early-life offer), because churn is highest in the first 12 months.
- Review fiber-optic pricing and service quality, and bundle tech support or online security with fiber plans.
- Treat three or more support calls as an alert: assign a senior agent and resolve the problem on the first contact.
- Encourage electronic-check customers to switch to automatic payment (bank transfer or credit card) with a small incentive.
- Score all customers every month with the model and send the offer only to the highest-risk group, using the profit curve to decide the cut-off.
- Track churn rate, recall of the model and campaign profit every month and retrain the model when performance falls.

# 9. Uses of the Project

- Churn monitoring: one place to see churn rate by contract, tenure, service and payment method.
- Prediction: a risk score for every customer.

- Campaign planning: decides how many customers to contact and expected profit.
- Reporting: charts and tables for management presentations.
- Learning tool: a complete, reproducible analytics workflow on GitHub.

## 10. Applications

| Stakeholder | How the project can be applied |
|---|---|
| Telecom and broadband companies | Predict and reduce customer churn; target retention offers. |
| OTT and subscription businesses | Identify at-risk subscribers from usage and billing data. |
| Banks and insurance companies | Predict customers likely to close accounts or lapse policies. |
| Marketing and CRM teams | Prioritise retention campaigns by expected profit. |
| Students and analysts | Template for any classification and business-impact analysis project. |

## 11. Limitations

- The dataset is simulated, so patterns are cleaner than in real data and real model accuracy would be lower.
- Customer lifetime value is approximated by twelve months of billing; real value depends on margin, expected stay and upsell.
- The campaign cost (Rs. 500) and success rate (30%) are assumptions; the true values should be measured with a small pilot.
- The model shows association, not proof of cause; for example, fiber customers may churn because of price, quality or competitors.
- A single time period is analysed; no seasonality or changes in customer behaviour over time are included.

## 12. Conclusion and Future Scope

The analysis shows that churn is concentrated among new, month-to-month, fiber-optic customers who call support often and pay by electronic check. The overall churn rate is 26.2%, but it ranges from 3.1% for two-year contracts to 42.9% for month-to-month contracts. Logistic Regression predicts churn with a ROC-AUC of 0.854, and a risk-based retention campaign aimed at the top 40% of customers is clearly more profitable than contacting everybody.

Future scope:

- Replace the simulated file with a real customer export from the billing system and CRM.
- Handle class imbalance and tune models with cross-validation and hyper-parameter search.
- Add usage data (call minutes, data used) and customer-satisfaction scores as predictors.
- Run an A/B test of the retention offer to measure the true success rate.
- Build a Streamlit dashboard that shows live risk scores and campaign profit.

# 13. GitHub Deployment

The project folder is already structured as a repository. To deploy it on GitHub:

Fig 7.1 GitHub Deployment Flow




*Fig 7.1 GitHub deployment flow*

**Step 1.** Create a new empty repository on github.com (for example `customer-churn-analysis`) without adding a README. **Step 2.** Open a terminal inside the project folder and run:

```
git init
git add .
git commit -m "Initial commit: Customer churn analysis and prediction"
git branch -M main
git remote add origin https://github.com/<your-username>/customer-churn-analysis.git
git push -u origin main
```

**Step 3.** Keep this PDF report in the docs/ folder. **Step 4.** Add the repository link and topics (python, data-analytics, business-analytics, machine-learning, churn-prediction) on the repository page. **Step 5.** Anyone can reproduce the results with:

```
pip install -r requirements.txt
cd src
python generate_data.py
python analysis.py
python diagrams.py
python build_report.py
```

**GitHub repository link:** _____

# 14. References


- Kotler, P. and Keller, K. L. Marketing Management. Pearson.
- Provost, F. and Fawcett, T. (2013). Data Science for Business. O'Reilly Media.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). The Elements of Statistical Learning. Springer.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python (Pandas). Proceedings of the 9th Python in Science Conference.
- Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32.
- Scikit-learn documentation: https://scikit-learn.org
- GitHub Docs: https://docs.github.com

# Appendix - Complete Source Code

## A.1 src/generate_data.py

```
"""
generate_data.py
Creates a simulated telecom customer dataset (5,000 customers) for churn analysis.
Churn is generated from a logistic model of realistic drivers (contract type, tenure, internet
service, support calls, payment method ...). Replace data/telecom_churn.csv with a real CRM export
to run the same pipeline on real data.
"""
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
N = 5000

gender = rng.choice(["Male", "Female"], N)
senior = rng.binomial(1, 0.16, N)
partner = rng.choice(["Yes", "No"], N, p=[0.48, 0.52])
dependents = np.where(partner == "Yes", rng.choice(["Yes", "No"], N, p=[0.40, 0.60]), rng.choice(["Yes", "No"], N, p=[0.12,
    0.88]))
contract = rng.choice(["Month-to-month", "One year", "Two year"], N, p=[0.55, 0.21, 0.24])
tenure = np.where(contract == "Month-to-month", rng.gamma(1.6, 12, N),
                  np.where(contract == "One year", rng.gamma(4.0, 9, N), rng.gamma(5.0, 11, N)))
tenure = np.clip(np.round(tenure), 1, 72).astype(int)
internet = rng.choice(["DSL", "Fiber optic", "No"], N, p=[0.34, 0.44, 0.22])
has_net = internet != "No"
def addon(p):
    return np.where(has_net, rng.choice(["Yes", "No"], N, p=[p, 1 - p]), "No internet")
online_security = addon(0.30)
tech_support = addon(0.30)
streaming = addon(0.48)
payment = rng.choice(["Electronic check", "Mailed check", "Bank transfer", "Credit card"], N, p=[0.34, 0.20, 0.23, 0.23])
paperless = rng.choice(["Yes", "No"], N, p=[0.59, 0.41])
support_calls = np.clip(rng.poisson(np.where(internet == "Fiber optic", 1.6, 1.1), N), 0, 8)

base = np.where(internet == "Fiber optic", 950, np.where(internet == "DSL", 600, 250))
monthly = base + 90 * (online_security == "Yes") + 90 * (tech_support == "Yes") + 130 * (streaming == "Yes") + rng.normal(0, 40,
     N)
monthly = np.round(np.clip(monthly, 150, None), 0)
total = np.round(monthly * tenure * rng.normal(1.0, 0.03, N), 0)

z = (1.35 * (contract == "Month-to-month") - 0.55 * (contract == "One year") - 1.2 * (contract == "Two year")
     - 0.035 * tenure + 0.75 * (internet == "Fiber optic") + 0.50 * (tech_support == "No")
     + 0.40 * (online_security == "No") + 0.65 * (payment == "Electronic check") + 0.25 * (paperless == "Yes")
     + 0.30 * senior + 0.28 * support_calls + 0.0010 * (monthly - 700))
lo, hi = -10, 10
for _ in range(60):                       # choose intercept so that average churn is about 26.5 percent
    mid = (lo + hi) / 2
    if (1 / (1 + np.exp(-(z + mid)))).mean() > 0.265: hi = mid
    else: lo = mid
p = 1 / (1 + np.exp(-(z + mid)))
churn = rng.binomial(1, p)

df = pd.DataFrame({
    "customer_id": [f"CUS{10000 + i}" for i in range(N)], "gender": gender, "senior_citizen": senior,
    "partner": partner, "dependents": dependents, "tenure_months": tenure, "contract": contract,
    "internet_service": internet, "online_security": online_security, "tech_support": tech_support,
    "streaming": streaming, "payment_method": payment, "paperless_billing": paperless,
    "support_calls": support_calls, "monthly_charges": monthly, "total_charges": total,
    "churn": np.where(churn == 1, "Yes", "No")})

# a few missing values to demonstrate cleaning
df.loc[rng.choice(df.index, 14, replace=False), "total_charges"] = np.nan
df.loc[rng.choice(df.index, 9, replace=False), "monthly_charges"] = np.nan
df.to_csv("../data/telecom_churn.csv", index=False)
print(df.shape, "churn rate:", round((df.churn == "Yes").mean() * 100, 1))
```

## A.2 src/analysis.py

```
"""
analysis.py
Cleaning -> EDA -> churn prediction models -> evaluation -> retention-campaign profit analysis -> charts
for the Customer Churn Analysis and Prediction project.
"""
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
                             roc_curve, confusion_matrix)

OUT = "../outputs/charts/"
NAVY, TEAL, ORANGE, GREY, GOLD = "#1F3A5F", "#2A9D8F", "#E76F51", "#8D99AE", "#E9C46A"
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False})

# ---------- 1. Load & clean ----------
df = pd.read_csv("../data/telecom_churn.csv")
raw_missing = {k: int(v) for k, v in df.isna().sum().items() if v > 0}
```

```
df["monthly_charges"] = df.monthly_charges.fillna(df.groupby("internet_service").monthly_charges.transform("median"))
df["total_charges"] = df.total_charges.fillna((df.monthly_charges * df.tenure_months).round())
df["churn_flag"] = (df.churn == "Yes").astype(int)
df["tenure_group"] = pd.cut(df.tenure_months, [0, 12, 24, 48, 72], labels=["0-12", "13-24", "25-48", "49-72"])
df.to_csv("../data/telecom_churn_clean.csv", index=False)

N = len(df); churn_rate = df.churn_flag.mean() * 100
rev_lost = df.loc[df.churn_flag == 1, "monthly_charges"].sum()
def rate_by(col):
    g = df.groupby(col, observed=True).churn_flag.agg(["mean", "count"])
    g["mean"] *= 100
    return g
by_contract, by_tenure, by_net = rate_by("contract"), rate_by("tenure_group"), rate_by("internet_service")
by_pay, by_support, by_ts = rate_by("payment_method"), rate_by("support_calls"), rate_by("tech_support")
by_senior = rate_by("senior_citizen")

# ---------- 2. EDA charts ----------
# C1 overall + contract
fig, axs = plt.subplots(1, 2, figsize=(7.0, 3.3))
cnt = df.churn.value_counts()
axs[0].bar(["Stayed", "Churned"], [cnt["No"], cnt["Yes"]], color=[TEAL, ORANGE])
for i, v in enumerate([cnt["No"], cnt["Yes"]]):
    axs[0].text(i, v + 40, f"{v:,}\n({v / N * 100:.1f}%)", ha="center", fontsize=8)
axs[0].set_ylim(0, cnt.max() * 1.2); axs[0].set_title("Customers by churn status", fontsize=10)
axs[1].bar(by_contract.index, by_contract["mean"], color=[ORANGE, GOLD, TEAL])
for i, v in enumerate(by_contract["mean"]): axs[1].text(i, v + 0.6, f"{v:.1f}%", ha="center", fontsize=8)
axs[1].set_ylim(0, by_contract["mean"].max() * 1.2); axs[1].set_title("Churn rate by contract (%)", fontsize=10)
axs[1].tick_params(axis="x", labelsize=8)
fig.suptitle("Fig 4.1 Overall Churn and Contract Type", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c1_overall_contract.png", dpi=200); plt.close(fig)

# C2 tenure
fig, ax = plt.subplots(figsize=(5.6, 3.2))
ax.bar(by_tenure.index.astype(str), by_tenure["mean"], color=[ORANGE, GOLD, TEAL, NAVY])
for i, v in enumerate(by_tenure["mean"]): ax.text(i, v + 0.6, f"{v:.1f}%", ha="center", fontsize=8)
ax.set_xlabel("Tenure group (months)"); ax.set_ylabel("Churn rate (%)"); ax.set_ylim(0, by_tenure["mean"].max() * 1.2)
ax.set_title("Fig 4.2 Churn Rate by Customer Tenure", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c2_tenure.png", dpi=200); plt.close(fig)

# C3 internet + tech support
fig, axs = plt.subplots(1, 2, figsize=(7.0, 3.2))
axs[0].bar(by_net.index, by_net["mean"], color=[TEAL, ORANGE, GREY][:len(by_net)])
for i, v in enumerate(by_net["mean"]): axs[0].text(i, v + 0.6, f"{v:.1f}%", ha="center", fontsize=8)
axs[0].set_title("By internet service (%)", fontsize=10); axs[0].set_ylim(0, by_net["mean"].max() * 1.2)
ts = by_ts.loc[["Yes", "No", "No internet"]]
axs[1].bar(["With tech\nsupport", "No tech\nsupport", "No\ninternet"], ts["mean"], color=[TEAL, ORANGE, GREY])
for i, v in enumerate(ts["mean"]): axs[1].text(i, v + 0.6, f"{v:.1f}%", ha="center", fontsize=8)
axs[1].set_title("By tech support (%)", fontsize=10); axs[1].set_ylim(0, ts["mean"].max() * 1.2)
fig.suptitle("Fig 4.3 Churn Rate by Service Features", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c3_services.png", dpi=200); plt.close(fig)

# C4 monthly charges
fig, ax = plt.subplots(figsize=(5.8, 3.2))
ax.hist([df[df.churn_flag == 0].monthly_charges, df[df.churn_flag == 1].monthly_charges], bins=22,
        color=[TEAL, ORANGE], label=["Stayed", "Churned"], density=True)
ax.set_xlabel("Monthly charges (Rs.)"); ax.set_ylabel("Density"); ax.legend(frameon=False)
ax.set_title("Fig 4.4 Monthly Charges: Churned vs Stayed", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c4_monthly_charges.png", dpi=200); plt.close(fig)

# C5 support calls + payment method
fig, axs = plt.subplots(1, 2, figsize=(7.0, 3.3))
sc = by_support[by_support["count"] >= 30]
axs[0].plot(sc.index.astype(str), sc["mean"], marker="o", color=NAVY)
axs[0].set_xlabel("Support calls"); axs[0].set_title("By support calls (%)", fontsize=10)
pm = by_pay.sort_values("mean")
axs[1].barh(pm.index, pm["mean"], color=[TEAL, TEAL, GOLD, ORANGE])
for i, v in enumerate(pm["mean"]): axs[1].text(v + 0.5, i, f"{v:.1f}%", va="center", fontsize=8)
axs[1].set_xlim(0, pm["mean"].max() * 1.2); axs[1].set_title("By payment method (%)", fontsize=10)
axs[1].tick_params(axis="y", labelsize=8)
fig.suptitle("Fig 4.5 Churn Rate by Support Calls and Payment Method", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c5_calls_payment.png", dpi=200); plt.close(fig)

# C6 correlation
num = df[["tenure_months", "monthly_charges", "total_charges", "support_calls", "senior_citizen", "churn_flag"]]
cc = num.corr().round(2)
fig, ax = plt.subplots(figsize=(5.4, 4.2))
im = ax.imshow(cc.values, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(cc))); ax.set_xticklabels(cc.columns, rotation=40, ha="right", fontsize=8)
ax.set_yticks(range(len(cc))); ax.set_yticklabels(cc.index, fontsize=8)
for i in range(len(cc)):
    for j in range(len(cc)): ax.text(j, i, f"{cc.values[i, j]:.2f}", ha="center", va="center", fontsize=7)
fig.colorbar(im, fraction=0.045); ax.set_title("Fig 4.6 Correlation Matrix (numeric columns)", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c6_corr.png", dpi=200); plt.close(fig)

# ---------- 3. Models ----------
feat = ["senior_citizen", "tenure_months", "support_calls", "monthly_charges", "gender", "partner", "dependents", "contract",
        "internet_service", "online_security", "tech_support", "streaming", "payment_method", "paperless_billing"]
X = pd.get_dummies(df[feat], drop_first=True).astype(float)
y = df.churn_flag
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
models = {"Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
          "Random Forest": RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=10, random_state=42),
          "Gradient Boosting": GradientBoostingClassifier(random_state=42)}
res, prob, pred = {}, {}, {}
for n, m in models.items():
    m.fit(X_tr, y_tr); pr = m.predict_proba(X_te)[:, 1]; pd_ = (pr >= 0.5).astype(int)
    prob[n], pred[n] = pr, pd_
    res[n] = {"Accuracy": accuracy_score(y_te, pd_), "Precision": precision_score(y_te, pd_), "Recall": recall_score(y_te, pd_),
              "F1": f1_score(y_te, pd_), "AUC": roc_auc_score(y_te, pr)}
res = {k: {m: round(float(v), 3) for m, v in d.items()} for k, d in res.items()}
best = max(res, key=lambda k: res[k]["AUC"])
```

```
baseline_acc = round(float(1 - y_te.mean()), 3)

fig, ax = plt.subplots(figsize=(5.2, 4.2))
for (n, pr), c in zip(prob.items(), [NAVY, TEAL, ORANGE]):
    f, t, _ = roc_curve(y_te, pr); ax.plot(f, t, color=c, label=f"{n} (AUC {res[n]['AUC']:.3f})")
ax.plot([0, 1], [0, 1], color=GREY, ls="--"); ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate")
ax.legend(frameon=False, fontsize=8, loc="lower right"); ax.set_title("Fig 5.1 ROC Curves of the Models", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c7_roc.png", dpi=200); plt.close(fig)

cm = confusion_matrix(y_te, pred[best])
fig, ax = plt.subplots(figsize=(4.0, 3.6))
ax.imshow(cm, cmap="Blues")
for i in range(2):
    for j in range(2): ax.text(j, i, f"{cm[i, j]}", ha="center", va="center", fontsize=14, color="white" if cm[i, j] > cm.max()
        / 2 else "black")
ax.set_xticks([0, 1]); ax.set_xticklabels(["Stay", "Churn"]); ax.set_yticks([0, 1]); ax.set_yticklabels(["Stay", "Churn"])
ax.set_xlabel("Predicted"); ax.set_ylabel("Actual"); ax.set_title(f"Fig 5.2 Confusion Matrix ({best})", fontsize=10)
fig.tight_layout(); fig.savefig(OUT + "c8_confusion.png", dpi=200); plt.close(fig)

rf = models["Random Forest"]
imp = pd.Series(rf.feature_importances_, index=X.columns)
groups = {}
for f in feat:
    cols = [c for c in X.columns if c == f or c.startswith(f + "_")]
    groups[f.replace("_", " ").title()] = imp[cols].sum()
gi = pd.Series(groups).sort_values(ascending=False).head(8).iloc[::-1]
fig, ax = plt.subplots(figsize=(5.8, 3.4))
ax.barh(gi.index, gi.values * 100, color=NAVY)
for i, v in enumerate(gi.values * 100): ax.text(v + 0.4, i, f"{v:.1f}%", va="center", fontsize=8)
ax.set_xlim(0, gi.max() * 100 * 1.15); ax.set_xlabel("Importance (%)")
ax.set_title("Fig 5.3 What Drives Churn? (Random Forest importance)", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c9_importance.png", dpi=200); plt.close(fig)
top_drivers = {k: round(float(v) * 100, 1) for k, v in gi.iloc[::-1].items()}

lr = models["Logistic Regression"]
coef = pd.Series(lr.steps[-1][1].coef_[0], index=X.columns).sort_values()
odds = np.exp(coef)

# ---------- 4. Retention campaign profit ----------
COST, SUCCESS = 500, 0.30                       # Rs. per customer contacted, probability that offer saves a churner
test = X_te.copy(); test["prob"] = prob[best]; test["churn"] = y_te.values; test["value"] = test.monthly_charges * 12
test = test.sort_values("prob", ascending=False).reset_index(drop=True)
gain = test.churn * test.value * SUCCESS - COST
cum = gain.cumsum()
k_best = int(cum.idxmax()) + 1
profit_best, profit_all = float(cum.max()), float(cum.iloc[-1])
fig, ax = plt.subplots(figsize=(6.0, 3.4))
ax.plot((np.arange(len(cum)) + 1) / len(cum) * 100, cum / 1e3, color=TEAL)
ax.axvline(k_best / len(cum) * 100, color=ORANGE, ls="--", lw=1)
ax.axhline(0, color="black", lw=0.7)
ax.text(k_best / len(cum) * 100 + 2, cum.max() / 1e3 * 0.5, f"Best: contact top {k_best / len(cum) * 100:.0f}%\nprofit Rs.
    {profit_best / 1e3:,.0f}K", color=ORANGE, fontsize=8.5)
ax.set_xlabel("Share of customers contacted (highest risk first, %)"); ax.set_ylabel("Net profit (Rs. thousand)")
ax.set_title("Fig 6.1 Retention Campaign Profit Curve (test set)", fontsize=10.5)
fig.tight_layout(); fig.savefig(OUT + "c10_profit.png", dpi=200); plt.close(fig)

# ---------- 5. Save ----------
out = {"n": N, "churn_rate": churn_rate, "churned": int(df.churn_flag.sum()), "raw_missing": raw_missing,
       "avg_tenure": float(df.tenure_months.mean()), "avg_monthly": float(df.monthly_charges.mean()),
       "monthly_rev_lost": float(rev_lost), "annual_rev_lost": float(rev_lost * 12),
       "by_contract": by_contract.round(1).reset_index().values.tolist(), "by_tenure":
           by_tenure.round(1).reset_index().values.tolist(),
       "by_net": by_net.round(1).reset_index().values.tolist(), "by_pay": by_pay.round(1).reset_index().values.tolist(),
       "by_ts": by_ts.round(1).reset_index().values.tolist(), "by_support": by_support.round(1).reset_index().values.tolist(),
       "by_senior": by_senior.round(1).reset_index().values.tolist(),
       "avg_monthly_churn": float(df[df.churn_flag == 1].monthly_charges.mean()), "avg_monthly_stay": float(df[df.churn_flag ==
           0].monthly_charges.mean()),
       "models": res, "best": best, "baseline_acc": baseline_acc, "test_n": len(y_te), "train_n": len(y_tr),
       "cm": cm.tolist(), "drivers": top_drivers,
       "odds": {k: round(float(v), 2) for k, v in odds.sort_values().items()},
       "campaign": {"cost": COST, "success": SUCCESS, "k_best": k_best, "k_best_pct": k_best / len(cum) * 100,
                    "profit_best": profit_best, "profit_all": profit_all, "churners_in_test": int(test.churn.sum()),
                    "caught_at_best": int(test.churn[:k_best].sum())},
       "corr_churn": cc["churn_flag"].to_dict()}
json.dump(out, open("../outputs/results.json", "w"), indent=2, default=str)
print(json.dumps({k: out[k] for k in ["n", "churn_rate", "raw_missing", "models", "best", "baseline_acc", "drivers", "campaign",
     "annual_rev_lost"]}, indent=1))
print(by_contract.round(1)); print(by_tenure.round(1)); print(by_net.round(1)); print(by_ts.round(1)); print(by_pay.round(1));
    print(by_support.round(1))
print(odds.sort_values().tail(6).round(2)); print(odds.sort_values().head(6).round(2))
```

## A.3 src/diagrams.py

```
"""diagrams.py - architecture, churn-funnel and GitHub-flow diagrams for the report."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

OUT = "../outputs/charts/"
NAVY, TEAL, ORANGE, LIGHT, GREY, GOLD = "#1F3A5F", "#2A9D8F", "#E76F51", "#EAF4F3", "#8D99AE", "#E9C46A"


def box(ax, x, y, w, h, text, fc=NAVY, tc="white", fs=8.5):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.12", fc=fc, ec="none"))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", color=tc, fontsize=fs, fontweight="bold", linespacing=1.3)


def arrow(ax, x1, y1, x2, y2):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>", mutation_scale=12, color=GREY, lw=1.6))
```

```
# 1. architecture
fig, ax = plt.subplots(figsize=(7.0, 5.2)); ax.set_xlim(0, 10); ax.set_ylim(0, 8); ax.axis("off")
box(ax, 0.3, 6.4, 2.6, 1.2, "1. DATA SOURCE\nCustomer records\n(5,000 customers)", TEAL)
box(ax, 3.7, 6.4, 2.6, 1.2, "2. DATA CLEANING\nFix missing charges\nencode features", NAVY)
box(ax, 7.1, 6.4, 2.6, 1.2, "3. EDA\nChurn by contract,\ntenure, service", NAVY)
arrow(ax, 2.9, 7.0, 3.7, 7.0); arrow(ax, 6.3, 7.0, 7.1, 7.0)
box(ax, 7.1, 4.2, 2.6, 1.2, "4. TRAIN / TEST SPLIT\n80% train\n20% test", NAVY); arrow(ax, 8.4, 6.4, 8.4, 5.4)
box(ax, 3.7, 4.2, 2.6, 1.2, "5. CHURN MODELS\nLogistic Regression\nRF, Gradient Boosting", ORANGE); arrow(ax, 7.1, 4.8, 6.3,
    4.8)
box(ax, 0.3, 4.2, 2.6, 1.2, "6. EVALUATION\nAUC, Recall, F1,\nConfusion matrix", NAVY); arrow(ax, 3.7, 4.8, 2.9, 4.8)
box(ax, 0.3, 2.0, 2.6, 1.2, "7. DRIVERS\nFeature importance,\nodds ratios", GOLD, tc="black"); arrow(ax, 1.6, 4.2, 1.6, 3.2)
box(ax, 3.7, 2.0, 2.6, 1.2, "8. RETENTION\nCAMPAIGN PROFIT\nWho to contact", TEAL); arrow(ax, 2.9, 2.6, 3.7, 2.6)
box(ax, 7.1, 2.0, 2.6, 1.2, "9. GITHUB\nDEPLOYMENT\nCode + README + report", NAVY); arrow(ax, 6.3, 2.6, 7.1, 2.6)
ax.add_patch(FancyBboxPatch((0.3, 0.3), 9.4, 1.0, boxstyle="round,pad=0.02,rounding_size=0.12", fc=LIGHT, ec=TEAL, lw=1.2))
ax.text(5, 0.8, "Tools: Python | Pandas | NumPy | Matplotlib | Scikit-learn | Git & GitHub", ha="center", va="center",
    fontsize=9, color=NAVY, fontweight="bold")
ax.set_title("Fig 3.1 System Architecture / Workflow of the Project", fontsize=10.5, color=NAVY, pad=6)
fig.tight_layout(); fig.savefig(OUT + "d1_architecture.png", dpi=200); plt.close(fig)

# 2. business mechanism
fig, ax = plt.subplots(figsize=(7.0, 3.6)); ax.set_xlim(0, 10); ax.set_ylim(0, 5.4); ax.axis("off")
steps = [("CUSTOMER\nDATA", NAVY), ("CHURN RISK\nSCORE (0-1)", TEAL), ("RANK BY\nRISK", GOLD), ("RETENTION\nOFFER", ORANGE),
    ("CUSTOMER\nSAVED", NAVY)]
for i, (s, c) in enumerate(steps):
    x = 0.1 + i * 2.0
    box(ax, x, 3.3, 1.7, 1.3, s, c, tc="black" if c == GOLD else "white", fs=8)
    if i < 4: arrow(ax, x + 1.7, 3.95, x + 2.0, 3.95)
box(ax, 0.3, 1.5, 4.4, 0.9, "Net gain = saved customers x annual revenue - offer cost", TEAL, fs=8)
box(ax, 5.3, 1.5, 4.4, 0.9, "Contact only customers whose expected gain is positive", ORANGE, fs=8)
ax.text(5, 0.6, "Recall = churners found / all churners   |   Precision = churners found / customers flagged", ha="center",
    fontsize=8.5, fontweight="bold", color=NAVY)
ax.set_title("Fig 3.2 Working Mechanism: From Risk Score to Retention Profit", fontsize=10.5, color=NAVY, pad=6)
fig.tight_layout(); fig.savefig(OUT + "d2_mechanism.png", dpi=200); plt.close(fig)

# 3. github flow
fig, ax = plt.subplots(figsize=(7.0, 1.9)); ax.set_xlim(0, 10); ax.set_ylim(0, 2.6); ax.axis("off")
steps = ["Local project\nfolder", "git init\ngit add .", "git commit\n-m message", "Create repo\non GitHub", "git push\norigin
    main"]
for i, s in enumerate(steps):
    x = 0.1 + i * 2.0
    box(ax, x, 0.7, 1.7, 1.2, s, NAVY if i % 2 == 0 else TEAL, fs=8)
    if i < 4: arrow(ax, x + 1.7, 1.3, x + 2.0, 1.3)
ax.set_title("Fig 7.1 GitHub Deployment Flow", fontsize=10.5, color=NAVY, pad=2)
fig.tight_layout(); fig.savefig(OUT + "d3_github.png", dpi=200); plt.close(fig)
print("ok")
```

## A.4 src/build_report.py

```
"""
build_report.py - builds the PDF project report from results.json, the CSV data and the charts.
Run after generate_data.py, analysis.py and diagrams.py.   Requires: reportlab, pillow
"""
import json, textwrap
from PIL import Image
import pandas as pd
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_JUSTIFY, TA_CENTER
from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
                                Image as RLImage, PageBreak, Preformatted, KeepTogether)

R = json.load(open("../outputs/results.json"))
CH = "../outputs/charts/"
NAME, ROLL, SEC = "Abhishek Kumar", "252737000", "B"
NAVY, TEAL, ORANGE, LIGHT = colors.HexColor("#1F3A5F"), colors.HexColor("#2A9D8F"), colors.HexColor("#E76F51"),
    colors.HexColor("#EAF4F3")

body = ParagraphStyle("b", fontName="Helvetica", fontSize=10, leading=14.5, alignment=TA_JUSTIFY, spaceAfter=6)
bul = ParagraphStyle("bul", parent=body, leftIndent=14, bulletIndent=3, spaceAfter=3, alignment=0)
h1 = ParagraphStyle("h1", fontName="Helvetica-Bold", fontSize=16, leading=20, textColor=NAVY, spaceBefore=6, spaceAfter=8)
h2 = ParagraphStyle("h2", fontName="Helvetica-Bold", fontSize=11.5, leading=15, textColor=TEAL, spaceBefore=8, spaceAfter=4)
cap = ParagraphStyle("cap", fontName="Helvetica-Oblique", fontSize=8.5, leading=11, alignment=TA_CENTER,
    textColor=colors.HexColor("#555555"), spaceAfter=8)
cell = ParagraphStyle("cell", fontName="Helvetica", fontSize=8.5, leading=10.5)
cellb = ParagraphStyle("cellb", parent=cell, fontName="Helvetica-Bold", textColor=colors.white)
code = ParagraphStyle("code", fontName="Courier", fontSize=7.6, leading=9.2, backColor=colors.HexColor("#F2F4F7"), leftIndent=4,
     spaceAfter=6)
tiny = ParagraphStyle("tiny", fontName="Courier", fontSize=6, leading=7.2)

def P(t): return Paragraph(t, body)
def B(t): return Paragraph(t, bul, bulletText="\u2022")
def H1(t): return Paragraph(t, h1)
def H2(t): return Paragraph(t, h2)
def IMG(f, w_mm):
    w, h = Image.open(CH + f).size
    return [RLImage(CH + f, width=w_mm * mm, height=w_mm * mm * h / w)]
def FIG(f, w_mm, c): return KeepTogether(IMG(f, w_mm) + [Paragraph(c, cap)])
def T(head, rows, widths, align=None):
    data = [[Paragraph(str(h), cellb) for h in head]] + [[Paragraph(str(c), cell) for c in r] for r in rows]
    t = Table(data, colWidths=[w * mm for w in widths], repeatRows=1)
    st = [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#BFC5CC")),
          ("VALIGN", (0, 0), (-1, -1), "MIDDLE"), ("TOPPADDING", (0, 0), (-1, -1), 3), ("BOTTOMPADDING", (0, 0), (-1, -1), 3)]
    return t if not st else (t.setStyle(TableStyle(st)) or t)
def CODE(lines): return Preformatted("\n".join(lines), code)

f0 = lambda x: f"{x:,.0f}"; f1 = lambda x: f"{x:,.1f}"
rate = lambda lst, key: next(r[1] for r in lst if r[0] == key)
cnt = lambda lst, key: next(r[2] for r in lst if r[0] == key)
```

```
M = R["models"]; best = R["best"]; cm = R["cm"]; C = R["campaign"]; D = R["drivers"]; O = R["odds"]
tn, fp, fn_, tp = cm[0][0], cm[0][1], cm[1][0], cm[1][1]

def footer(c, d):
    c.saveState(); c.setFont("Helvetica", 7.5); c.setFillColor(colors.HexColor("#777777"))
    c.drawString(20 * mm, 10 * mm, f"{NAME} | Roll No. {ROLL} | Sec {SEC} | NIET Business School")
    c.drawRightString(190 * mm, 10 * mm, f"Page {d.page}")
    c.setStrokeColor(TEAL); c.line(20 * mm, 13 * mm, 190 * mm, 13 * mm); c.restoreState()

doc = BaseDocTemplate("../docs/Customer_Churn_Analysis_Report.pdf", pagesize=A4, leftMargin=20 * mm, rightMargin=20 * mm,
                      topMargin=18 * mm, bottomMargin=20 * mm, title="Customer Churn Analysis and Prediction",
                      author=f"{NAME} (Roll No. {ROLL})")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(20 * mm, 18 * mm, 170 * mm, 261 * mm, id="f")], onPage=footer)])
S = []

# ---------- cover ----------
S += [Spacer(1, 35 * mm), Paragraph("NIET BUSINESS SCHOOL", ParagraphStyle("c1", fontName="Helvetica-Bold", fontSize=26,
    alignment=TA_CENTER, textColor=NAVY, leading=32)),
      Paragraph("Post Graduate Diploma in Management (PGDM)", ParagraphStyle("c2", fontName="Helvetica", fontSize=12,
          alignment=TA_CENTER, textColor=colors.HexColor("#555555"), leading=18)),
      Spacer(1, 8 * mm), Paragraph("BUSINESS ANALYTICS &amp;amp; DATA ANALYTICS | PROJECT REPORT", ParagraphStyle("c3",
          fontName="Helvetica-Bold", fontSize=12, alignment=TA_CENTER, textColor=TEAL, leading=16)),
      Spacer(1, 14 * mm), Paragraph("PROJECT TITLE", ParagraphStyle("c4", fontName="Helvetica-Bold", fontSize=9,
          alignment=TA_CENTER, leading=12)),
      Paragraph("Customer Churn Analysis and Prediction for a Telecom Company", ParagraphStyle("c5", fontName="Helvetica-Bold",
          fontSize=22, alignment=TA_CENTER, textColor=NAVY, leading=28)),
      Spacer(1, 18 * mm)]
cov = Table([["Submitted by", NAME], ["Roll Number", ROLL], ["Section", SEC], ["Programme", "PGDM"], ["College", "NIET Business
    School"], ["Submission", "October 2026"]], colWidths=[55 * mm, 80 * mm])
cov.setStyle(TableStyle([("FONTNAME", (0, 0), (-1, -1), "Helvetica-Bold"), ("FONTSIZE", (0, 0), (-1, -1), 11), ("LINEBELOW", (0,
     0), (-1, -1), 0.5, colors.HexColor("#BFC5CC")),
                         ("TOPPADDING", (0, 0), (-1, -1), 7), ("BOTTOMPADDING", (0, 0), (-1, -1), 7)]))
S += [cov, Spacer(1, 18 * mm), Paragraph("Tools: Python | Pandas | NumPy | Matplotlib | Scikit-learn | Git &amp;amp; GitHub",
    ParagraphStyle("c6", fontName="Helvetica-Bold", fontSize=9.5, alignment=TA_CENTER)), PageBreak()]

# ---------- contents ----------
toc = ["Title and Introduction", "Problem Statement, Objectives, Purpose and Scope", "Tools Used, Project Structure,
    Architecture Diagram and Working Mechanism",
       "Dataset Description", "Work Done (Step-by-Step Methodology)", "Exploratory Data Analysis and Data Interpretation",
         "Churn Prediction Model and Interpretation",
       "Retention Campaign Scenario and Recommendations", "Uses of the Project", "Applications", "Limitations", "Conclusion and
         Future Scope", "GitHub Deployment", "References", "Appendix - Complete Source Code"]
S += [H1("Table of Contents"), T(["No.", "Section"], [[i + 1 if i < 14 else "A", t] for i, t in enumerate(toc)], [20, 150]),
    PageBreak()]

# ---------- 1 ----------
S += [H1("1. Title and Introduction"), H2("1.1 Project Title"), P("Customer Churn Analysis and Prediction for a Telecom
    Company"), H2("1.2 Introduction"),
      P("Churn means a customer stops using a company's service. For subscription businesses such as telecom, broadband and OTT,
          winning a new customer usually costs much more than keeping an existing one, so every customer who leaves is a direct
          loss of future revenue. Business analytics helps a company answer three questions: who is leaving, why are they
          leaving, and which customers should be contacted first with a retention offer."),
      P(f"This project analyses {f0(R['n'])} telecom customers. It cleans the data, studies how churn changes with contract
          type, tenure, internet service, support calls, payment method and charges, builds three machine-learning models to
          predict which customers will churn, identifies the main drivers of churn, and finally converts the model into a
          business decision by estimating the profit of a targeted retention campaign."),
      P("<b>Important note on data:</b> the dataset is simulated. It was generated by a seeded Python script (generate_data.py)
          in which churn depends on realistic drivers, so that the full analytics pipeline can be demonstrated and reproduced.
          The method is reusable: replacing the CSV with a real customer export from a CRM or billing system will run the same
          pipeline. Findings describe this dataset and should not be read as real market statistics.")]
# ---------- 2 ----------
S += [H1("2. Problem Statement, Objectives, Purpose and Scope"), H2("2.1 Problem Statement"),
      P(f"A telecom company loses {f1(R['churn_rate'])}% of its customers, which is about Rs. {f1(R['annual_rev_lost'] / 1e5)}
          lakh of yearly billing in this dataset, but it does not know which customers are about to leave or why. Contacting
          every customer with an offer is expensive. The question this project answers is: which customer groups churn most,
          what drives churn, can churn be predicted in advance, and which customers should be targeted to make a retention
          campaign profitable?"),
      H2("2.2 Objectives"),
      B("To clean the customer data and calculate the churn rate."), B("To find how churn varies with contract, tenure,
          services, support calls, payment method and charges."),
      B("To build and compare churn prediction models using standard evaluation measures."), B("To identify the most important
          drivers of churn."),
      B("To estimate the profit of a targeted retention campaign and decide how many customers to contact."), B("To package the
          project with documentation for deployment on GitHub."),
      H2("2.3 Purpose"), P("The purpose is to turn customer records into decisions: who to protect, what to fix and where to
          spend retention budget. It also demonstrates the complete analytics lifecycle: data preparation, exploratory analysis,
           predictive modelling, evaluation, business interpretation and deployment."),
      H2("2.4 Scope"), P("The scope covers 5,000 customers, 14 predictor columns and the churn status. Customer lifetime value
          is approximated by twelve months of billing. Network quality data, competitor offers and customer satisfaction surveys
           are outside the scope.")]
# ---------- 3 ----------
S += [PageBreak(), H1("3. Tools, Structure, Architecture and Mechanism"), H2("3.1 Tools and Technologies Used"),
      T(["Tool / Library", "Purpose in this project"], [["Python 3", "Main programming language for the entire pipeline"],
          ["Pandas", "Loading the CSV, cleaning, grouping and summarising"],
        ["NumPy", "Simulated data generation and numerical operations"], ["Matplotlib", "All charts and diagrams"],
        ["Scikit-learn", "Train/test split, scaling, Logistic Regression, Random Forest, Gradient Boosting and evaluation
            metrics"],
        ["Git and GitHub", "Version control and public hosting of the project"], ["ReportLab", "Generation of this PDF
            report"]], [45, 125]), Spacer(1, 4),
      H2("3.2 Project Structure (Repository Layout)"),
      CODE(["customer-churn-analysis/", "|-- data/", "|   |-- telecom_churn.csv         raw dataset (5,000 customers)", "|   `--
          telecom_churn_clean.csv   cleaned data with extra columns",
           "|-- src/", "|   |-- generate_data.py          builds the dataset", "|   |-- analysis.py              cleaning,
               EDA, models, evaluation, campaign profit, charts",
           "|   |-- diagrams.py              architecture / mechanism diagrams", "|   `-- build_report.py           builds the
               PDF report", "|-- outputs/", "|   |-- charts/                  all figures (PNG)",
           "|   `-- results.json             metrics and summary numbers", "|-- docs/", "|   `--
               Customer_Churn_Analysis_Report.pdf", "|-- requirements.txt", "|-- .gitignore", "`-- README.md"]),
      H2("3.3 System Architecture Diagram"), FIG("d1_architecture.png", 135, "Fig 3.1  System architecture / workflow of the
          project"),
      P("The pipeline runs from raw customer data to cleaning, exploratory analysis, model training, evaluation, driver
```

```
        analysis, campaign profit estimation and deployment."),
      H2("3.4 Working Mechanism"), FIG("d2_mechanism.png", 140, "Fig 3.2  Working mechanism: from risk score to retention
          profit"),
      P("<b>How the mechanism works.</b> The trained model gives every customer a churn risk score between 0 and 1. Customers
          are ranked from highest to lowest risk. A retention offer (a discount, a plan upgrade or a call from the retention
          team) costs money, so it should be sent only where the expected benefit is larger than the cost. The expected benefit
          is the chance of saving the customer multiplied by the revenue the customer would otherwise take away."),
      T([["Measure", "Formula", "Meaning"], [["Churn rate", "Churned customers / All customers x 100", "Share of customers
          lost"],
        ["Accuracy", "Correct predictions / All predictions", "Overall correctness"], ["Precision", "True churners flagged / All
            customers flagged", "How reliable a flag is"],
        ["Recall", "True churners flagged / All real churners", "How many churners are found"], ["F1-score", "2 x Precision x
            Recall / (Precision + Recall)", "Balance of precision and recall"],
        ["ROC-AUC", "Area under the ROC curve", "Ability to rank churners above stayers (0.5 = random, 1 = perfect)"]], [30, 70,
            70])]
# ---------- 4 ----------
S += [PageBreak(), H1("4. Dataset Description"), P("The dataset has 5,000 customers and 17 columns (one row per customer). Extra
     columns (churn flag and tenure group) are added during analysis."),
      T([["Column", "Type", "Description"], [["customer_id", "Text", "Unique customer ID (CUS10000 onwards)"], ["gender,
          senior_citizen, partner, dependents", "Category", "Customer profile"],
        ["tenure_months", "Number", "Months the customer has been with the company"], ["contract", "Category", "Month-to-month /
            One year / Two year"],
        ["internet_service", "Category", "DSL / Fiber optic / No"], ["online_security, tech_support, streaming", "Category",
            "Add-on services (Yes / No / No internet)"],
        ["payment_method", "Category", "Electronic check / Mailed check / Bank transfer / Credit card"], ["paperless_billing",
            "Category", "Yes / No"],
        ["support_calls", "Number", "Customer-care calls in the recent period"], ["monthly_charges", "Number", "Monthly bill in
            INR"], ["total_charges", "Number", "Total billed so far in INR"],
        ["churn", "Target", "Yes if the customer left, No if the customer stayed"]], [58, 22, 90]),
      H2("4.1 Data Quality and Cleaning"),
      P(f"The raw file had {R['raw_missing'].get('monthly_charges', 0)} missing values in monthly_charges and
          {R['raw_missing'].get('total_charges', 0)} missing values in total_charges. Missing monthly charges were filled with
          the median of the customer's internet-service group, because the bill depends strongly on the service type. Missing
          total charges were rebuilt as monthly charges multiplied by tenure. No duplicate customer IDs were found."),
      H2("4.2 Overall Summary"),
      T([["Metric", "Value", "Metric", "Value"], [["Customers", f0(R["n"]), "Churned customers", f0(R["churned"])], ["Churn
          rate", f"{f1(R['churn_rate'])}%", "Average tenure", f"{f1(R['avg_tenure'])} months"],
        ["Average monthly bill", f"Rs. {f0(R['avg_monthly'])}", "Monthly billing lost to churn", f"Rs. {f1(R['monthly_rev_lost']
            / 1e5)} lakh"],
        ["Yearly billing at risk", f"Rs. {f1(R['annual_rev_lost'] / 1e5)} lakh", "Train / test split", f"{f0(R['train_n'])} /
            {f0(R['test_n'])}"]], [48, 37, 48, 37])]
# ---------- 5 ----------
S += [PageBreak(), H1("5. Work Done (Step-by-Step Methodology)"),
      H2("Step 1 - Topic selection and problem definition"), P("Customer churn was chosen because it is one of the most common
          business-analytics problems and has a clear financial meaning."),
      H2("Step 2 - Data creation"), P("A seeded script simulates 5,000 customers. Contract, services, payment method and support
           calls are drawn with realistic proportions, and churn is generated from a logistic equation in which month-to-month
          contracts, short tenure, fiber service, electronic-check payment, more support calls and no tech support raise the
          churn probability. The intercept is tuned so that overall churn is close to 26%."),
      H2("Step 3 - Data cleaning"), P("Missing charges were filled from service-group medians and tenure (Section 4.1). Tenure
          was also grouped into 0-12, 13-24, 25-48 and 49-72 months."),
      H2("Step 4 - Exploratory analysis"), P("Churn rate was calculated for every important feature and six charts were produced
           (Section 6)."),
      H2("Step 5 - Model building"), P("Categorical columns were one-hot encoded. The data was split 80/20 with stratification
          so that the churn share is the same in both parts."),
      CODE(["X = pd.get_dummies(df[feat], drop_first=True).astype(float)", "X_tr, X_te, y_tr, y_te = train_test_split(X, y,
          test_size=0.2, stratify=y, random_state=42)",
            "models = {'Logistic Regression': make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),",
            "          'Random Forest': RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=10,
              random_state=42),",
            "          'Gradient Boosting': GradientBoostingClassifier(random_state=42)}"]),
      H2("Step 6 - Evaluation"), P("Models were compared on the unseen test customers using accuracy, precision, recall,
          F1-score and ROC-AUC. The model with the highest ROC-AUC was selected (Section 7)."),
      H2("Step 7 - Business scenario"), P("The selected model's risk scores were used to rank customers and to calculate the
          profit of a retention campaign for different numbers of contacted customers (Section 8)."),
      H2("Step 8 - Documentation and deployment"), P("README, requirements and .gitignore were prepared so the project can be
          pushed to GitHub (Section 13).")]
# ---------- 6 ----------
bc, bt, bn, bp, bs = R["by_contract"], R["by_tenure"], R["by_net"], R["by_pay"], R["by_support"]
S += [PageBreak(), H1("6. Exploratory Data Analysis and Data Interpretation"), H2("6.1 Overall Churn and Contract Type"),
     FIG("c1_overall_contract.png", 150, "Fig 4.1  Overall churn and churn rate by contract type"),
      P(f"Out of {f0(R['n'])} customers, {f0(R['churned'])} ({f1(R['churn_rate'])}%) left. Contract type is the clearest
          dividing line: month-to-month customers churn at {f1(rate(bc, 'Month-to-month'))}%, compared with {f1(rate(bc, 'One
          year'))}% for one-year and only {f1(rate(bc, 'Two year'))}% for two-year contracts. A month-to-month customer is about
           {rate(bc, 'Month-to-month') / rate(bc, 'Two year'):.0f} times as likely to leave as a two-year customer, because
          there is no commitment holding them."),
      H2("6.2 Tenure"), FIG("c2_tenure.png", 110, "Fig 4.2  Churn rate by customer tenure"),
      P(f"Churn falls steadily with tenure: {f1(rate(bt, '0-12'))}% in the first year, {f1(rate(bt, '13-24'))}% in the second
          year and {f1(rate(bt, '49-72'))}% for customers older than four years. The first twelve months are the most dangerous
          period, so onboarding and early-life offers have the highest return."),
      H2("6.3 Internet Service and Tech Support"), FIG("c3_services.png", 150, "Fig 4.3  Churn rate by internet service and tech
           support"),
      P(f"Fiber-optic customers churn at {f1(rate(bn, 'Fiber optic'))}%, almost twice the {f1(rate(bn, 'DSL'))}% of DSL
          customers, even though they pay the highest bills; price or service problems on the premium plan are a likely cause.
          Customers with internet but without tech support churn at {f1(rate(R['by_ts'], 'No'))}% versus {f1(rate(R['by_ts'],
          'Yes'))}% with tech support. Customers without internet churn least ({f1(rate(bn, 'No'))}%), since they have a simple,
           cheap phone-only plan."),
      H2("6.4 Monthly Charges"), FIG("c4_monthly_charges.png", 120, "Fig 4.4  Distribution of monthly charges for churned and
          stayed customers"),
      P(f"Churned customers pay more on average (Rs. {f0(R['avg_monthly_churn'])} per month) than customers who stayed (Rs.
          {f0(R['avg_monthly_stay'])}). High bills are partly a result of the fiber plans, so price alone should not be blamed;
          price and service quality act together."),
      H2("6.5 Support Calls and Payment Method"), FIG("c5_calls_payment.png", 150, "Fig 4.5  Churn rate by number of support
          calls and by payment method"),
      P(f"Each extra support call is a warning sign: churn is {f1(rate(bs, 0))}% for customers with no calls, {f1(rate(bs, 3))}%
           for three calls and {f1(rate(bs, 4))}% for four calls. Customers paying by electronic check churn at {f1(rate(bp,
          'Electronic check'))}%, against about 23% for bank transfer, credit card and mailed cheque; automatic payment
          customers are more settled."),
      H2("6.6 Correlation Matrix"), FIG("c6_corr.png", 105, "Fig 4.6  Correlation matrix of numeric columns"),
      P(f"Tenure has the strongest negative correlation with churn ({R['corr_churn']['tenure_months']:.2f}): longer-staying
          customers are far less likely to leave. Support calls ({R['corr_churn']['support_calls']:.2f}) and monthly charges
```

```
        ({R['corr_churn']['monthly_charges']:.2f}) are positively related to churn. Total charges is strongly linked to tenure
         because it is monthly charges accumulated over time, so it is not used as a predictor.")]
# ---------- 7 ----------
S += [PageBreak(), H1("7. Churn Prediction Model and Interpretation"),
      P(f"Three models were trained on {f0(R['train_n'])} customers and tested on {f0(R['test_n'])} unseen customers. A model
          that simply predicts that nobody churns would be {R['baseline_acc'] * 100:.1f}% accurate, so accuracy alone is not
          enough; recall and ROC-AUC matter more."),
      T(["Model", "Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"], [[k, v["Accuracy"], v["Precision"], v["Recall"],
          v["F1"], v["AUC"]] for k, v in M.items()], [48, 24, 24, 24, 24, 26]), Spacer(1, 6),
      P(f"<b>Selected model: {best}</b> (ROC-AUC = {M[best]['AUC']}). It ranks a random churner above a random stayer about
          {M[best]['AUC'] * 100:.0f}% of the time, and at the default 0.5 cut-off it finds {M[best]['Recall'] * 100:.0f}% of
          churners, with {M[best]['Precision'] * 100:.0f}% of its churn flags being correct."),
      P("<b>Why Logistic Regression wins here.</b> In this simulated data the drivers of churn add up in a smooth way, which is
          exactly what logistic regression assumes, so a simple model matches or beats the complex ones and is also easy to
          explain. On real customer data, where effects interact, tree-based models often do better; the pipeline compares them
          automatically, so the best model is always chosen from the numbers."),
      H2("7.1 ROC Curves and Confusion Matrix"), FIG("c7_roc.png", 100, "Fig 5.1  ROC curves of the three models"),
          FIG("c8_confusion.png", 75, f"Fig 5.2  Confusion matrix of {best} on the test customers"),
      P(f"Of {f0(R['test_n'])} test customers, the model correctly identified {tp} churners and {tn} stayers. It missed {fn_}
          churners (false negatives) and wrongly flagged {fp} stayers (false positives). For retention, missing a churner is
          usually more costly than contacting a customer who would have stayed, so the cut-off can be lowered to catch more
          churners; Section 8 chooses it using profit."),
      H2("7.2 What Drives Churn"), FIG("c9_importance.png", 115, "Fig 5.3  Importance of each feature group (Random Forest)"),
      P(f"Tenure ({D['Tenure Months']}%) and contract type ({D['Contract']}%) together explain more than half of the model's
          decisions, followed by monthly charges ({D['Monthly Charges']}%) and internet service ({D['Internet Service']}%).
          Support calls, payment method and tech support have smaller but real effects."),
      P(f"Logistic-regression odds ratios agree: a two-year contract multiplies the odds of churn by {O['contract_Two year']}
          and a one-year contract by {O['contract_One year']} (compared with month-to-month), fiber-optic service raises the
          odds by {O['internet_service_Fiber optic']} times, electronic-check payment by {O['payment_method_Electronic check']}
          times, and one more standard deviation of support calls by {O['support_calls']} times. One standard deviation of
          tenure multiplies the odds by {O['tenure_months']}.")]
# ---------- 8 ----------
S += [PageBreak(), H1("8. Retention Campaign Scenario and Recommendations"),
      P(f"<b>Scenario:</b> a retention offer costs Rs. {C['cost']} per customer contacted. It is assumed to save a churning
          customer with a probability of {C['success'] * 100:.0f}%, and a saved customer is worth twelve months of billing. Test
           customers are ranked by predicted churn risk and contacted from the highest risk downwards. Net profit = (saved
          customers x yearly billing) - (customers contacted x Rs. {C['cost']}})."),
      H2("8.1 Scenario Result"), FIG("c10_profit.png", 120, "Fig 6.1  Net profit of the retention campaign by share of customers
           contacted"),
      P(f"Profit rises while the most risky customers are contacted, peaks, and then falls as low-risk customers (who would not
          have left anyway) are added. The best choice is to contact the top {C['k_best_pct']:.0f}% of customers ({C['k_best']}
          of {f0(R['test_n'])}), which reaches {C['caught_at_best']} of the {C['churners_in_test']} churners and gives a net
          profit of about Rs. {f0(C['profit_best'] / 1e3)} thousand on the test group. Contacting everyone would give only Rs.
          {f0(C['profit_all'] / 1e3)} thousand, and doing nothing gives zero. Targeting by risk score therefore raises campaign
          profit by about {(C['profit_best'] / C['profit_all'] - 1) * 100:.0f}% over a blanket campaign. This is an estimate
          based on the assumed cost and success rate, not a guarantee."),
      H2("8.2 Recommendations"),
      B("Move month-to-month customers to one- or two-year plans with a discount or a free add-on; this is the single largest
          lever."),
      B("Build a first-year programme (welcome call, usage tips, early-life offer), because churn is highest in the first 12
          months."),
      B("Review fiber-optic pricing and service quality, and bundle tech support or online security with fiber plans."),
      B("Treat three or more support calls as an alert: assign a senior agent and resolve the problem on the first contact."),
      B("Encourage electronic-check customers to switch to automatic payment (bank transfer or credit card) with a small
          incentive."),
      B("Score all customers every month with the model and send the offer only to the highest-risk group, using the profit
          curve to decide the cut-off."),
      B("Track churn rate, recall of the model and campaign profit every month and retrain the model when performance falls.")]
# ---------- 9-12 ----------
S += [H1("9. Uses of the Project"), B("Churn monitoring: one place to see churn rate by contract, tenure, service and payment
    method."), B("Prediction: a risk score for every customer."),
      B("Campaign planning: decides how many customers to contact and expected profit."), B("Reporting: charts and tables for
          management presentations."), B("Learning tool: a complete, reproducible analytics workflow on GitHub."),
      H1("10. Applications"),
      T(["Stakeholder", "How the project can be applied"], [["Telecom and broadband companies", "Predict and reduce customer
          churn; target retention offers."], ["OTT and subscription businesses", "Identify at-risk subscribers from usage and
          billing data."],
        ["Banks and insurance companies", "Predict customers likely to close accounts or lapse policies."], ["Marketing and CRM
            teams", "Prioritise retention campaigns by expected profit."],
        ["Students and analysts", "Template for any classification and business-impact analysis project."]], [60, 110]),
      H1("11. Limitations"),
      B("The dataset is simulated, so patterns are cleaner than in real data and real model accuracy would be lower."),
      B("Customer lifetime value is approximated by twelve months of billing; real value depends on margin, expected stay and
          upsell."),
      B("The campaign cost (Rs. 500) and success rate (30%) are assumptions; the true values should be measured with a small
          pilot."),
      B("The model shows association, not proof of cause; for example, fiber customers may churn because of price, quality or
          competitors."),
      B("A single time period is analysed; no seasonality or changes in customer behaviour over time are included."),
      H1("12. Conclusion and Future Scope"),
      P(f"The analysis shows that churn is concentrated among new, month-to-month, fiber-optic customers who call support often
          and pay by electronic check. The overall churn rate is {f1(R['churn_rate'])}%, but it ranges from {f1(rate(bc, 'Two
          year'))}% for two-year contracts to {f1(rate(bc, 'Month-to-month'))}% for month-to-month contracts. {best} predicts
          churn with a ROC-AUC of {M[best]['AUC']}, and a risk-based retention campaign aimed at the top {C['k_best_pct']:.0f}%
          of customers is clearly more profitable than contacting everybody."),
      P("Future scope:"), B("Replace the simulated file with a real customer export from the billing system and CRM."),
          B("Handle class imbalance and tune models with cross-validation and hyper-parameter search."),
      B("Add usage data (call minutes, data used) and customer-satisfaction scores as predictors."), B("Run an A/B test of the
          retention offer to measure the true success rate."), B("Build a Streamlit dashboard that shows live risk scores and
          campaign profit.")]
# ---------- 13 ----------
S += [PageBreak(), H1("13. GitHub Deployment"), P("The project folder is already structured as a repository. To deploy it on
    GitHub:"), FIG("d3_github.png", 140, "Fig 7.1  GitHub deployment flow"),
      P("<b>Step 1.</b> Create a new empty repository on github.com (for example <font face='Courier'>customer-churn-
          analysis</font>) without adding a README. <b>Step 2.</b> Open a terminal inside the project folder and run:"),
      CODE(["git init", "git add .", 'git commit -m "Initial commit: Customer churn analysis and prediction"', "git branch -M
          main", "git remote add origin https://github.com/<your-username>/customer-churn-analysis.git", "git push -u origin
          main"]),
      P("<b>Step 3.</b> Keep this PDF report in the docs/ folder. <b>Step 4.</b> Add the repository link and topics (python,
          data-analytics, business-analytics, machine-learning, churn-prediction) on the repository page. <b>Step 5.</b> Anyone
          can reproduce the results with:"),
```

```
      CODE(["pip install -r requirements.txt", "cd src", "python generate_data.py", "python analysis.py", "python diagrams.py",
          "python build_report.py"]),
      Table([[Paragraph("<b>GitHub repository link:</b> _____", cell)]],
          colWidths=[170 * mm], style=[("BOX", (0, 0), (-1, -1), 0.8, colors.black), ("TOPPADDING", (0, 0), (-1, -1), 8),
          ("BOTTOMPADDING", (0, 0), (-1, -1), 8)]),
      H1("14. References"),
      B("Kotler, P. and Keller, K. L. Marketing Management. Pearson."), B("Provost, F. and Fawcett, T. (2013). Data Science for
          Business. O'Reilly Media."),
      B("Hastie, T., Tibshirani, R. and Friedman, J. (2009). The Elements of Statistical Learning. Springer."),
      B("Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12,
          2825-2830."),
      B("McKinney, W. (2010). Data Structures for Statistical Computing in Python (Pandas). Proceedings of the 9th Python in
          Science Conference."),
      B("Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32."), B("Scikit-learn documentation: https://scikit-
          learn.org"), B("GitHub Docs: https://docs.github.com")]
# ---------- appendix ----------
S += [PageBreak(), H1("Appendix - Complete Source Code")]
for i, fn in enumerate(["generate_data.py", "analysis.py", "diagrams.py", "build_report.py"]):
    S.append(H2(f"A.{i + 1} src/{fn}")); out = []
    for ln in open(fn).read().rstrip().split("\n"):
        ind = len(ln) - len(ln.lstrip())
        out += textwrap.wrap(ln, 128, subsequent_indent=" " * (ind + 4), replace_whitespace=False, drop_whitespace=False) or
            [""]
    S.append(Preformatted("\n".join(out), tiny))
doc.build(S); print("report built")
```